

MODULE 2

Inheritance and Interfacing

Inheritance, Method Overriding, Annotations, Static members, Inner Classes, Abstract Classes, Final members and classes, The Object Class, Interfaces, Package Fundamentals, Reflections

2.1 INTRODUCTION TO INHERITANCE:

Inheritance in Java is a core OOP concept that allows a class to acquire properties and behaviours from another class. Inheritance is a mechanism in which one object acquires all the properties and behaviors of parent object. Inheritance represents the IS-A relationship, also known as parent- child relationship. It helps in creating a new class from an existing class, promoting code reusability and better organization.

- A subclass can reuse the fields and methods of the parent class without rewriting the code
- A subclass can add its own fields and methods or modify existing ones to extend functionality.

Why use inheritance in java:

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Syntax of Java Inheritance

```
class Base_class_name
```

```
{    //Access_specifier data declaration
```

```
    //Access_specifier member_function(parameter_list);
```

```
}  
  
class Subclass_name extends Superclass_name  
  
{    //methods and fields  
  
}
```

Inheritance:

- One class can acquire the properties of another class.
- A class that is inherited is called a superclass.
- The class that does the inheriting is called a subclass.

Therefore, a subclass is a specialized version of a superclass. It inherits all of the instance variables and methods defined by the superclass and adds its own, unique elements.

To inherit a class, simply incorporate the definition of one class into another by using the **extends** keyword. The following program creates a superclass called A and a subclass called B. The keyword extends is used to create a subclass of A.

// Create a Base class or superclass.

```
class A  
{    int i, j;  
    void showij()  
    {    System.out.println("i and j: " + i + " " + j);  
    }  
}
```

// Create a subclass by extending class A.

```
class B extends A  
{ int k;  
    void showk()  
    {    System.out.println("k: " + k);  
    }  
}
```

```
        void sum()
        {           System.out.println("i+j+k: " + (i+j+k));
        }
    }

class SimpleInheritance {
public static void main(String args[])
{
    A superOb = new A();

    B subOb = new B(); // The superclass may be used by itself.

    superOb.i = 10;

    superOb.j = 20;

    System.out.println("Contents of superOb: ");

    superOb.showij();

    System.out.println();

    /* The subclass has access to all public members of its superclass. */

    subOb.i = 7; subOb.j = 8; subOb.k = 9;

    System.out.println("Contents of subOb: ");

    subOb.showij();

    subOb.showk();

    System.out.println();

    System.out.println("Sum of i, j and k in subOb:");

    subOb.sum();

}

}
```

output: Contents of superOb: i and j: 10 20

Contents of subOb: i and j: 7 8 k: 9

Sum of i, j and k in subOb: i+j+k: 24

The subclass B includes all of the members of its superclass, A. This is why subOb can access i and j and call showij(). Also, inside sum(), i and j can be referred to directly, as if they were part of B.

Even though A is a superclass for B, it is also a completely independent, stand-alone class. Being a superclass for a subclass does not mean that the superclass cannot be used by itself.

A subclass can be a superclass for another subclass. Java does not support the inheritance of multiple super classes into a single subclass. But a subclass can become a superclass of another subclass. However, no class can be a superclass of itself.

Advantages of Inheritance in Java

1. One of the key benefits of inheritance is to minimize the amount of duplicate code in an application by sharing common code amongst several subclasses. Where equivalent code
2. Exists in two related classes, the hierarchy can usually be refactored to move the common code up to a mutual superclass. This also tends to result in a better organization of code and smaller, simpler compilation units.
3. Inheritance can also make application code more flexible to change because classes that inherit from a common superclass can be used interchangeably. If the return type of a method is super class
4. Reusability - facility to use public methods of base class without rewriting the same.
5. Extensibility - extending the base class logic as per business logic of the derived class.

Disadvantages of Inheritance in Java

- **Complexity:** Inheritance can make the code more complex and harder to understand. This is especially true if the inheritance hierarchy is deep or if multiple inheritances is used.
- **Tight Coupling:** Inheritance creates a tight coupling between the superclass and subclass, making it difficult to make changes to the superclass without affecting the subclass.

Types of Inheritance:

Java supports three primary types of inheritance with classes: single, multilevel, and hierarchical. Multiple and hybrid inheritance are only achievable through the use of interfaces, not classes.

Here are the types of inheritance in Java:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance

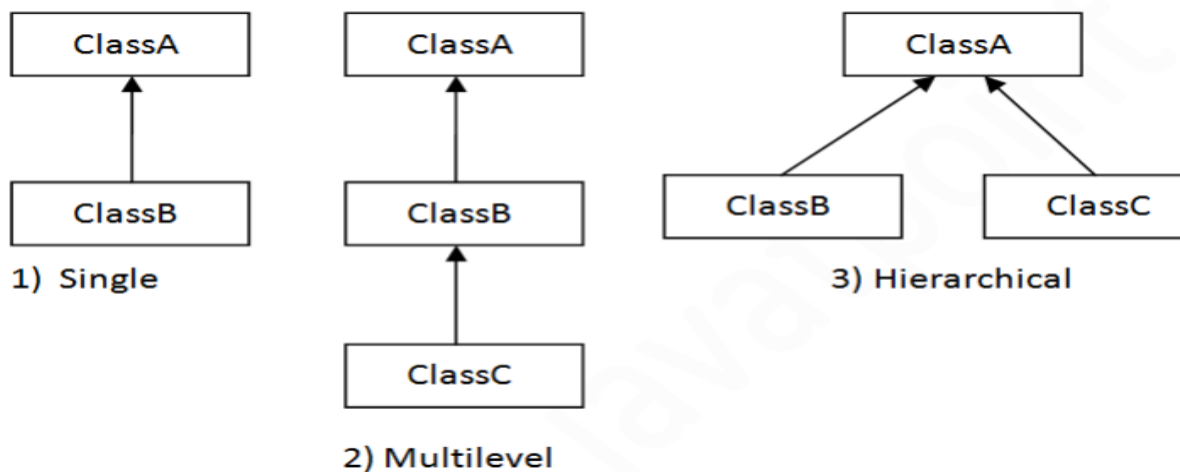


Figure: Types of Inheritance in Java

- **Single Inheritance:** In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance. In this type, a single subclass extends one superclass. This is the simplest form of inheritance and represents a direct "is-a" relationship (e.g., a Dog is an Animal).

Program Code:

```
//Super class  
class Vehicle {  
    Vehicle() {
```

```
        System.out.println("This is a Vehicle");
    }
}

// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

public class Test {
    public static void main(String[] args) {
        // Creating object of subclass invokes base class constructor
        Car obj = new Car();
    }
}
```

Output:

This is a Vehicle

This Vehicle is Car

- **Multilevel Inheritance:** In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes. This involves a chain of inheritance, where a class inherits from a class which itself inherits from another class (e.g., Puppy extends Dog extends Animal). The subclass has access to members of all ancestor classes in the chain.

Program Code:

```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
```

```
    }  
}  
class FourWheeler extends Vehicle {  
    FourWheeler() {  
        System.out.println("4 Wheeler Vehicles");  
    }  
}  
class Car extends FourWheeler {  
    Car() {  
        System.out.println("This 4 Wheeler Vehicle is a Car");  
    }  
}  
public class Geeks {  
    public static void main(String[] args) {  
        Car obj = new Car(); // Triggers all constructors in order  
    }  
}
```

Output:

This is a Vehicle

4 Wheeler Vehicles

This 4 Wheeler Vehicle is a Car

- **Hierarchical Inheritance:** In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. In this structure, multiple subclasses inherit from a single parent class (e.g., both Car and Bus extend the Vehicle class). This is useful for creating multiple specialized classes that share common base functionality. For example, cars and buses both are vehicle.

Program Code:

```
class Vehicle {  
    Vehicle() {  
        System.out.println("This is a Vehicle");  
    }  
}  
  
class Car extends Vehicle {  
    Car() {  
        System.out.println("This Vehicle is Car");  
    }  
}  
  
class Bus extends Vehicle {  
    Bus() {  
        System.out.println("This Vehicle is Bus");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Car obj1 = new Car();  
        Bus obj2 = new Bus();  
    }  
}
```

Output:

This is a Vehicle

This Vehicle is Car

This is a Vehicle

This Vehicle is Bus

Java does not support following inheritance types:

- **Multiple Inheritance:** Java does not support a class inheriting directly from more than one superclass to avoid ambiguity issues (known as the "diamond problem").
- **Hybrid Inheritance:** This is a combination of two or more types of inheritance. Since it can involve multiple inheritance paths, it is not directly supported with classes in Java but can be achieved using a combination of class inheritance and interfaces.

Member Access and Inheritance

Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as private.

// Create a superclass.

Class A

```
{    int i; // public by default private
    int j; // private to A
    void setij(int x, int y)
    {        i = x; j = y;
    }
```

} // A's j is not accessible here.

class B extends A

```
{    int total;
    void sum()
    {        total = i + j; // ERROR, j is not accessible here
    }
}
```

class Access

```
{    public static void main(String args[])
    {        B subOb = new B();
            subOb.setij(10, 12);
            subOb.sum();
    }
```

```
        System.out.println("Total is " + subOb.total);
    }
}
```

This program will not compile because the reference to `j` inside the `sum()` method of `B` causes an access violation. Since `j` is declared as `private`, it is only accessible by other members of its own class. Subclasses have no access to it.

Practical Example: The `Box` class developed will be extended to include a fourth component called `weight`. Thus, the new class will contain a box's width, height, depth, and weight.

// This program uses inheritance to extend `Box`.

class `Box`

```
{    double width; double height; double depth;
    // construct clone of an object
    Box(Box ob)
    { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
    // constructor used when all dimensions specified
    Box(double w, double h, double d)
    {    width = w; height = h; depth = d;
    }
    // constructor used when no dimensions specified
    Box()
    {    width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }
    // constructor used when cube is created
```

```
Box(double len)
{
    width = height = depth = len;
}

// compute and return volume
double volume()
{
    return width * height * depth;
}
}

// Here, Box is extended to include weight.
class BoxWeight extends Box
{
    double weight; // weight of box
    // constructor for BoxWeight
    BoxWeight(double w, double h, double d, double m)
    { width = w; height = h; depth = d; weight = m; }
}

class DemoBoxWeight
{
    public static void main(String args[])
    {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        double vol;
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println();
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);
    }
}
```

Output:

Volume of mybox1 is 3000.0 Weight of mybox1 is

34.3 Volume of mybox2 is

24.0 Weight of mybox2 is 0.076

A Superclass Variable Can Reference a Subclass Object

A reference variable of a superclass can be assigned a reference to any subclass derived from that superclass.

Class RefDemo

```
{    public static void main(String args[])
    {        BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
            Box plainbox = new Box();
            double vol;
            vol = weightbox.volume();
            System.out.println("Volume of weightbox is " + vol);
            System.out.println("Weight of weightbox is " + weightbox.weight);
            System.out.println();
            // assign BoxWeight reference to Box reference
            plainbox = weightbox;
            vol = plainbox.volume(); // OK, volume() defined in Box
            System.out.println("Volume of plainbox is " + vol);

            /* The following statement is invalid because plainbox does not define a weight
            member.*/
            System.out.println("Weight of plainbox is " + plainbox.weight);
        }
    }
```

Here, weightbox is a reference to BoxWeight objects, and plainbox is a reference to Box objects. Since BoxWeight is a subclass of Box, it is permissible to assign plainbox a reference to the weightbox object.

Super Keyword in Java

The super keyword in Java is a reference variable which is used to refer immediate parent class object. Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

1) super is used to refer immediate parent class instance variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

```
class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void printColor()
    {
        System.out.println(color);//prints color of Dog class
        System.out.println(super.color);//prints color of Animal class
    }
}
class TestSuper
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.printColor();
    }
}
```

Output:

black

white

In the above example, Animal and Dog both classes have a common property color. If we print color property, it will print the color of current class by default. To access the parent property, we need to use super keyword.

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}

class Dog extends Animal
{
    void eat()
    {
        System.out.println("eating bread...");
    }

    void bark()
    {
        System.out.println("barking...");
    }

    void work()
    {
        super.eat();
        bark();
    }
}
```

```
}  
class TestSuper  
{  
    public static void main(String args[])  
    {  
        Dog d=new Dog();  
        d.work();  
    }  
}
```

Output:

eating...

barking...

In the above example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local. To call the parent class method, we need to use super keyword.

3) super is used to invoke parent class constructor. The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

```
class Animal  
{  
    Animal()  
    {  
        System.out.println("animal is created");  
    }  
}  
class Dog extends Animal  
{  
    Dog()  
    {  
        super();  
        System.out.println("dog is created");  
    }  
}  
class TestSuper  
{  
    public static void main(String args[])  
    {  
        Dog d=new Dog();  
    }  
}
```

```
    }  
}
```

Output:

animal is created

dog is created

Creating a Multilevel Hierarchy: Given three classes called A, B, and C, C can be a subclass of B, which is a subclass of A. When this type of situation occurs, each subclass inherits all of the traits found in all of its super classes.

In this case, C inherits all aspects of B and A. In it, the subclass BoxWeight is used as a superclass to create the subclass called Shipment. Shipment inherits all of the traits of BoxWeight and Box, and adds a field called cost, which holds the cost of shipping such a parcel.

class Box

```
{    private double width; private double height;  
    private double depth;  
    // construct clone of an object  
    Box(Box ob)  
    {    width = ob.width; height = ob.height; depth = ob.depth;  
    }  
    Box(double w, double h, double d)  
    { width = w; height = h; depth = d; }  
    // constructor used when no dimensions specified  
    Box()  
    {    width = -1; // use -1 to indicate  
        height = -1; // an uninitialized  
        depth = -1; // box  
    }  
    Box(double len)
```



```
        {           width = height = depth = len; }
        double volume()
        {           return width * height * depth;
        }
    }

// Add weight.
class BoxWeight extends Box
{
    double weight;

    BoxWeight(BoxWeight ob)
    {
        super(ob);
        weight = ob.weight;
    }

    BoxWeight(double w, double h, double d, double m)
    {
        super(w, h, d);
        weight = m;
    }

    BoxWeight()
    {
        super();
        weight = -1;
    }

    BoxWeight(double len, double m)
    {
        super(len);
        weight = m;
    }
}

// Add shipping costs.
class Shipment extends BoxWeight
{
    double cost;

    Shipment(Shipment ob)
```

```
    {
        super(ob);
        cost = ob.cost;
    }
    Shipment(double w, double h, double d, double m, double c)
    {
        super(w, h, d, m); // call superclass constructor
        cost = c;
    }
    Shipment()
    {
        super(); cost = -1;
    }
    Shipment(double len, double m, double c)
    {
        super(len, m);
        cost = c;
    }
}

class DemoShipment
{
    public static void main(String args[])
    {
        Shipment shipment1 = new Shipment(10, 20, 15, 10, 3.41);
        Shipment shipment2 = new Shipment(2, 3, 4, 0.76, 1.28);
        double vol;
        vol = shipment1.volume();
        System.out.println("Volume of shipment1 is " + vol);
        System.out.println("Weight of shipment1 is " + shipment1.weight);
        System.out.println("Shipping cost: $" + shipment1.cost);
        System.out.println();
        vol = shipment2.volume();
        System.out.println("Volume of shipment2 is " + vol);
        System.out.println("Weight of shipment2 is " + shipment2.weight);
        System.out.println("Shipping cost: $" + shipment2.cost);
    }
}
```

```
}
```

Output : Volume of shipment1 is 3000.0

Weight of shipment1 is 10.0

Shipping cost: \$3.41

Volume of shipment2 is 24.0

Weight of shipment2 is 0.76

Shipping cost: \$1.28

When Constructors Are Called: Given a subclass called B and a superclass called A, is A's constructor called before B's, or vice versa? The answer is that in a class hierarchy, constructors are called in order of derivation, from superclass to subclass.

Further, since `super( )` must be the first statement executed in a subclass' constructor, this order is the same whether or not `super( )` is used.

```
class A
{
    A()
    {
        System.out.println("Inside A's constructor.");
    }
}

class B extends A
{
    B()
    {
        System.out.println("Inside B's constructor.");
    }
}

class C extends B
{
    C()
    {
        System.out.println("Inside C's constructor.");
    }
}
```

```
class CallingCons
{
    public static void main(String args[])
    {
        C c = new C();
    }
}
```

Output :

Inside A's constructor

Inside B's constructor

Inside C's constructor

2,2 METHOD OVERRIDING:

Method overriding allows us to achieve run-time polymorphism and is used for writing specific definitions of a subclass method that is already defined in the superclass. The method in superclass and overridden method in the subclass should have the same declaration signature such as parameters list, type, and return type.

When a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass.

Example: Program Code

```
class A
{
    int i, j;
    A(int a, int b)
    {
        i = a;
        j = b;
    }
    // display i and j
    void show()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}
```

```
}  
class B extends A  
{  
    int k  
    B(int a, int b, int c)  
    {  
        super(a, b); k = c;  
    }  
    void show()  
    {  
        System.out.println("k: " + k);  
    }  
}  
class Override  
{  
    public static void main(String args[])  
    {  
        B subOb = new B(1, 2, 3);  
        subOb.show(); // this calls show() in B  
    }  
}
```

Output: k: 3

The version of show() inside B overrides the version declared in A. To access the superclass version of an overridden method can be called using super.

```
class B extends A  
{  
    int k;  
    B(int a, int b, int c)  
    {  
        super(a, b); k = c;  
    }  
    void show()  
    {  
        super.show(); // this calls A's show()  
        System.out.println("k: " + k);  
    }  
}
```

Output: i and j: 1 2 k: 3

Here, super.show() calls the superclass version of show().

Method overriding occurs only when the names and the type signatures of the two methods are identical. If they are not, then the two methods are simply overloaded.

```
class A
{
    int i, j;
    A(int a, int b)
    {
        i = a; j = b;
    }
    // display i and j
    void show()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A.
class B extends A
{
    int k;
    B(int a, int b, int c)
    {
        super(a, b);
        k = c;
    }
    // overload show()
    void show(String msg)
    {
        System.out.println(msg + k);
    }
}

class Override
{
    public static void main(String args[])
    {
        B subOb = new B(1, 2, 3);
    }
}
```

```
        subOb.show("This is k: "); // this calls show() in B
        subOb.show(); // this calls show() in A
    }
}
```

The output produced by this program is shown here:

This is k: 3 i and j: 1 2

Advantage of Method Overriding

The main advantage of method overriding is that the class can give its own specific implementation to a inherited method without even modifying the parent class code.

This is helpful when a class has several child classes, so if a child class needs to use the parent class method, it can use it and the other classes that want to have different implementation can use overriding feature to make changes without touching the parent class code.

Dynamic Method Dispatch

Method Overriding is an example of runtime polymorphism. When a parent class reference points to the child class object then the call to the overridden method is determined at runtime, because during method call which method(parent class or child class) is to be executed is determined by the type of object. This process in which call to the overridden method is resolved at runtime is known as dynamic method dispatch.

```
class ABC
{
    //Overridden method
    public void disp()
    {
        System.out.println("disp() method of parent class");
    }
}

class Demo extends ABC
{
    //Overriding method
    public void disp()
```

```
        {      System.out.println("disp() method of Child class");
        }

    public void newMethod()
    {      System.out.println("new method of child class");
    }

    public static void main( String args[])
    {      /*When Parent class reference refers to the parent class object then in this case
            overridden method (the method of parent class) is called. */

        ABC obj = new ABC();

        obj.disp();

        /* When parent class reference refers to the child class object then the
        overriding method (method of child class) is called. This is called dynamic
        method dispatch and runtime polymorphism. */

        ABC obj2 = new Demo();

        obj2.disp();

    }
}
```

Output:

disp() method of parent class

disp() method of Child class

In the above example the call to the disp() method using second object (obj2) is runtime polymorphism (or dynamic method dispatch).

Note: In dynamic method dispatch the object can call the overriding methods of child class and all the non-overridden methods of base class but it cannot call the methods which are newly declared in the child class. In the above example the object obj2 is calling the disp(). However if you try to call the newMethod() method (which has been newly declared in Demo class) using obj2 then you would give compilation error .

Super keyword in Method Overriding

The `super` keyword is used for calling the parent class method/constructor. `super.myMethod()` calls the `myMethod()` method of base class while `super()` calls the constructor of base class. Let's see the use of `super` in method Overriding.

As we know that we we override a method in child class, then call to the method using child class object calls the overridden method. By using `super` we can call the overridden method as shown in the example below:

```
class ABC{
    public void myMethod()
    {
        System.out.println("Overridden method");
    }
}
class Demo extends ABC{
    public void myMethod(){
        //This will call the myMethod() of parent class
        super.myMethod();
        System.out.println("Overriding method");
    }
    public static void main( String args[]) {
        Demo obj = new Demo();
        obj.myMethod();
    }
}
```

Output:

Class ABC: mymethod()

Class Test: mymethod()

2.3 ANNOTATIONS IN JAVA:

Annotations in Java are a form of metadata that provide additional information about the program. They do not change the action of a compiled program but can be used by the compiler or runtime for processing. An annotation is a form of metadata that provides information about the code without changing its behavior.

Used by: Compiler, Build tools, Frameworks (Spring, Hibernate, JUnit)

- Annotations start with '@'.
- Annotations do not change the action of a compiled program.
- Annotations help to associate metadata (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
- Annotations are not pure comments as they can change the way a program is treated by the compiler. See below code for example.
- Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.

Hierarchy of Annotations in Java

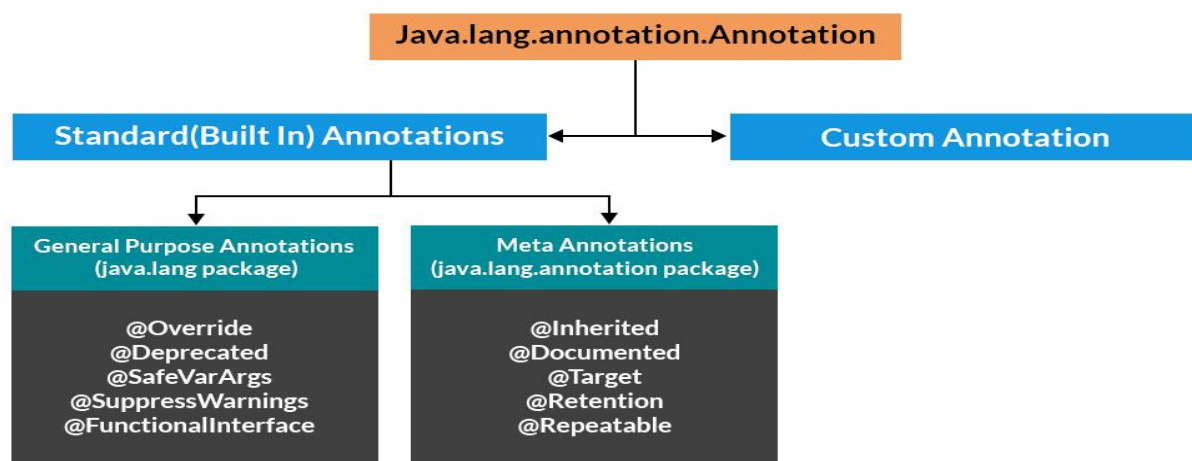


Figure: Hierarchy of Annotations in Java

Java includes several built-in annotations. Here are some of the most commonly used:

Annotation	Description
@Override	Indicates that a method overrides a method in a superclass
@Deprecated	Marks a method or class as outdated or discouraged from use
@SuppressWarnings	Tells the compiler to ignore certain warnings

@Override Annotation

The @Override annotation helps the compiler check that a method really overrides a method from a superclass. It's not required, but it's highly recommended because it helps catch errors.

In this example, we clearly indicate that we are overriding a method:

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Woof!");  
    }  
}
```

Output: Woof!

@Deprecated Annotation

The @Deprecated annotation warns developers not to use a method because it may be removed or replaced in the future:

```
public class Main {  
    @Deprecated  
    static void oldMethod() {  
        System.out.println("This method is outdated.");  
    }  
    public static void main(String[] args) {  
        oldMethod(); // This will show a warning in most IDEs  
    }  
}
```

Output: This method is outdated.

@SuppressWarnings Annotation

The @SuppressWarnings annotation tells the compiler to ignore specific warnings, like "unchecked" or "deprecation":

```
import java.util.ArrayList;  
public class Main {  
    @SuppressWarnings("unchecked")  
    public static void main(String[] args) {  
        ArrayList cars = new ArrayList();  
        cars.add("Volvo");  
        System.out.println(cars);  
    }  
}
```

Output: [Volvo]

Categories of Annotations: There are broadly 5 categories of annotations as listed:

- Marker Annotations
- Single value Annotations
- Full Annotations
- Type Annotations

- Repeating Annotations

1. Marker Annotations: Do not have any elements or values (just presence is enough). Used to signal the compiler or tools with metadata.

Example: @TestAnnotation()

2. Single value Annotations: Contain only one element (can use shorthand notation). When specifying value, you don't need to write the element name.

Example: @TestAnnotation("testing");

3. Full Annotations : Contain multiple elements as key-value pairs. All values must be provided unless defaults are defined.

Example: @TestAnnotation(owner="Rahul", value="Class Geeks")

4. Type Annotations: Introduced in Java 8 for annotating types (variables, generics, return types). Useful for stronger type checking and frameworks.

Declared with @Target(ElementType.TYPE_USE).

Example: Java Program to Demonstrate Type Annotation

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;
// Using target annotation to annotate a type
@Target(ElementType.TYPE_USE)
// Declaring a simple type annotation
@interface TypeAnnoDemo{
// Main class
public class GFG {
    // Main driver method
    public static void main(String[] args) {
        // Annotating the type of a string
        @TypeAnnoDemo String string = "I am annotated with a type annotation";
```

```
    System.out.println(string);
    abc();
}
// Annotating return type of a function
static @TypeAnnoDemo int abc() {
    System.out.println("This function's return type is annotated");
    return 0;
}
}
```

Output

I am annotated with a type annotation

This function's return type is annotated

5. Repeating Annotations: Allow the same annotation to be applied multiple times on a single element. Declared using @Repeatable with a container annotation.

Example: @Schedule(day="Monday") @Schedule(day="Tuesday").

Example: Java Program to Demonstrate a Repeatable Annotation

```
import java.lang.annotation.Annotation;
import java.lang.annotation.Repeatable;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.reflect.Method;
// Make Words annotation repeatable
@Retention(RetentionPolicy.RUNTIME)
@Repeatable(MyRepeatedAnnos.class)
@interface Words
{
    String word() default "Hello";
}
```

```
    int value() default 0;
}
// Create container annotation
@Retention(RetentionPolicy.RUNTIME)
@interface MyRepeatedAnnos
{
    Words[] value();
}
public class Main {
    // Repeat Words on newMethod
    @Words(word = "First", value = 1)
    @Words(word = "Second", value = 2)
    public static void newMethod()
    {
        Main obj = new Main();
        try {
            Class<?> c = obj.getClass();
            // Obtain the annotation for newMethod
            Method m = c.getMethod("newMethod");
            // Display the repeated annotation
            Annotation anno
                = m.getAnnotation(MyRepeatedAnnos.class);
            System.out.println(anno);
        }
        catch (NoSuchMethodException e) {
            System.out.println(e);
        }
    }
    public static void main(String[] args) { newMethod(); }
```

```
}
```

Output:

```
@MyRepeatedAnnos({@Words(value=1,word="First"),@Words(value=2,  
word="Second")})
```

From java.lang.annotation**1. @Documented**

Marks annotations to be included in generated Javadoc.

Improves API documentation clarity.

2. @Target

Specifies where the annotation can be applied (class, method, field, etc.).

Prevents misuse of annotations in wrong places.

3. @Inherited

Allows a subclass to inherit an annotation from its parent class.

Useful in frameworks where annotations define behavior across hierarchies.

User-defined (Custom) Annotation

User-defined annotations can be used to annotate program elements, i.e. variables, constructors, methods, etc. These annotations can be applied just before the declaration of an element (constructor, method, classes, etc).

Syntax

```
[Access Specifier] @interface<AnnotationName>
```

```
{
```

```
    DataType <Method Name>() [default value];
```

```
}
```


Do keep these certain points as rules for custom annotations before implementing user-defined annotations.

- AnnotationName is an interface.
- The parameter should not be associated with method declarations and throws clause should not be used with method declaration.
- Parameters will not have a null value but can have a default value.
- Default value is optional.
- The return type of method should be either primitive, enum, string, class name or array of primitive, enum, string or class name type.

Example: Java Program to Demonstrate User-defined Annotations

```
package source;
import java.lang.annotation.Documented;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
// User-defined annotation
@Documented
@Retention(RetentionPolicy.RUNTIME)
@ interface TestAnnotation
{
    String Developer() default "Rahul";
    String Expirydate();
} // will be retained at runtime
// Driver class that uses @TestAnnotation
public class Test
{
    @TestAnnotation(Developer="Rahul", Expirydate="01-10-2020")
    void fun1()
    {
```

```
        System.out.println("Test method 1");
    }
    @TestAnnotation(Developer="Anil", Expirydate="01-10-2021")
    void fun2()
    {
        System.out.println("Test method 2");
    }
    public static void main(String args[])
    {
        System.out.println("Hello");
    }
}
```

Output: Hello

2.4 STATIC MEMBERS:

In Java, static members are those which belongs to the class and you can access these members without instantiating the class.

The **static keyword** can be used with methods, fields, classes (inner/nested), blocks.

Static Methods – You can create a static method by using the keyword static. Static methods can access only static fields, methods. To access static methods there is no need to instantiate the class, you can do it just using the class name as –

Example

```
public class MyClass {
    public static void sample(){
        System.out.println("Hello");
    }
    public static void main(String args[]){
        MyClass.sample();
    }
}
```

```
}  
}
```

Output: Hello

Static Fields – You can create a static field by using the keyword static. The static fields have the same value in all the instances of the class. These are created and initialized when the class is loaded for the first time. Just like static methods you can access static fields using the class name (without instantiation).

Example

```
public class MyClass {  
    public static int data = 20;  
    public static void main(String args[]){  
        System.out.println(MyClass.data);  
    }  
}
```

Output: 20

Static Blocks – These are a block of codes with a static keyword. In general, these are used to initialize the static members. JVM executes static blocks before the main method at the time of class loading.

Example

```
public class MyClass {  
    static{  
        System.out.println("Hello this is a static block");  
    }  
    public static void main(String args[]){  
        System.out.println("This is main method");  
    }  
}
```

Output

Hello this is a static block

This is main method

2.5 INNER CLASS

An inner class is a class declared inside the body of another class. The inner class has access to all members (including private) of the outer class, but the outer class can access the inner class members only through an object of the inner class.

The most important type of nested class is the inner class. An **inner class** is a non-static nested class. It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.

Syntax:

```
class OuterClass {  
    // Outer class members  
    class InnerClass {  
        // Inner class members  
    }  
}
```

Example1: The following program illustrates how to define and use an inner class. The class named OuterClass has one instance method named display(), and defines one inner class called InnerClass.

```
public class OuterClass {  
    class InnerClass {  
        void display() {  
            System.out.println("Hello from Inner Class!");  
        }  
    }  
    public static void main(String[] args) {  
        OuterClass outer = new OuterClass();  
        InnerClass inner = outer.new InnerClass();  
    }  
}
```

```
        inner.display();
    }
}
```

Output: Hello from Inner Class!

Example 2: The following program illustrates how to define and use an inner class. The class named Outer has one instance variable named outer_x, one instance method named test(), and defines one inner class called Inner.

```
// Demonstrate an inner class.
class Outer {
    int outer_x = 100;
    void test() {
        Inner inner = new Inner();
        inner.display();
    }
    // this is an inner class
    class Inner {
        void display() {
            System.out.println("display: outer_x = " + outer_x);
        }
    }
}

class InnerClassDemo {
    public static void main(String args[]) {
        Outer outer = new Outer();
        outer.test();
    }
}
```

Output:

display: outer_x = 100

In the program, an inner class named **Inner** is defined within the scope of class **Outer**. Therefore, any code in class **Inner** can directly access the variable **outer_x**. An instance method named **display()** is defined inside **Inner**. This method displays **outer_x** on the standard output stream. The **main()** method of **InnerClassDemo** creates an instance of class **Outer** and invokes its **test()** method. That method creates an instance of class **Inner** and the **display()** method is called.

Note: An instance of Inner can be created only in the context of class Outer. The Java compiler generates an error message otherwise. In general, an inner class instance is often created by code within its enclosing scope, as the example does.

Although we have been focusing on inner classes declared as members within an outer class scope, it is possible to define inner classes within any block scope. For example, you can define a nested class within the block defined by a method or even within the body of a **for** loop, as this next program shows:

```
// Define an inner class within a for loop.
class Outer {
    int outer_x = 100;
    void test() {
        for(int i=0; i<10; i++) {
            class Inner {
                void display() {
                    System.out.println("display: outer_x = " + outer_x);
                }
            }
        }
        Inner inner = new Inner();
        inner.display();
    }
}
```

```
}  
}  
}  
class InnerClassDemo {  
    public static void main(String args[]) {  
        Outer outer = new Outer();  
        outer.test();  
    }  
}
```

Output:

```
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100  
display: outer_x = 100
```

While nested classes are not applicable to all situations, they are particularly helpful when handling events.

Features of Inner Classes

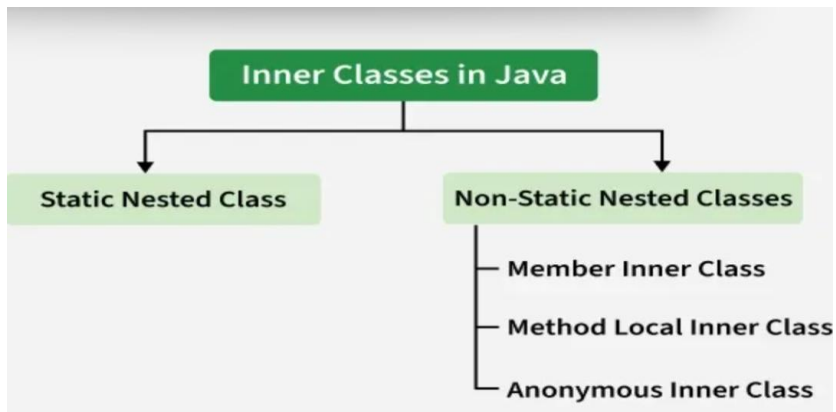
- **Encapsulation:** Inner classes can access private members of the outer class, providing better encapsulation.

- **Code Organization:** Logically groups classes that belong together, making code more readable.
- **Access to Outer Class:** Inner class instances have a reference to the outer class instance.
- **Namespace Management:** Helps avoid naming conflicts by nesting related classes.

Types of Inner Classes

Java supports four types of inner classes:

1. Member Inner Class
2. Method-Local Inner Class
3. Static Nested Class
4. Anonymous Inner Class



Member Inner Class

A member inner class is a non-static class defined at the member level of another class. It has access to all members of the outer class, including private members.

- Must be instantiated with an outer class instance.
- **Syntax:** `Outer.Inner inner = outer.new Inner();`


```
class Outer {  
    private int outerVar = 100;  
    // Member inner class  
    class Inner {  
        void display() {  
            System.out.println("Outer variable: " + outerVar);  
        }  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        // Creating inner class instance  
        Outer.Inner inner = new Outer().new Inner();  
        inner.display();  
    }  
}
```

Output:

Inside outerMethod

Inside innerMethod

Method Local Inner Class: A **method-local inner class** is defined inside a method of the outer class. It can only be instantiated within the method where it is defined.

Cannot access non-final local variables before Java 8. From Java 8 onwards, can access effectively final local variables. Cannot be declared as private, protected, static, or transient.

Can be declared as abstract or final, but not both.

```
class Outer {  
    void outerMethod() {  
        int x = 98; // Effectively final  
        System.out.println("Inside outerMethod");  
    }  
}
```

```
class Inner {  
    void innerMethod() {  
        System.out.println("x = " + x);  
    }  
}  
Inner inner = new Inner();  
inner.innerMethod();  
}  
}  
class Main {  
    public static void main(String[] args) {  
        Outer outer = new Outer();  
        outer.outerMethod();  
    }  
}
```

Output:

Inside outerMethod

x = 98

Static Nested Classes

A static nested class is a static class defined inside another class. It does not have access to instance members of the outer class but can access static members.

```
class Outer {  
    private static int staticVar = 50;  
    // Static nested class  
    static class Inner {  
        void display() {  
            System.out.println("Static variable: " + staticVar);  
        }  
    }  
}
```

```
    }  
}  
class Main {  
    public static void main(String[] args) {  
        Outer.Inner inner = new Outer.Inner();    // No need for outer class instance  
        inner.display();  
    }  
}
```

Output: Static variable: 50

2.6 ABSTRACT CLASSES

In Java, an abstract class is a class that cannot be instantiated and is designed to be extended by other classes. It is used to achieve partial abstraction, where some methods are implemented while others are left for subclasses to define. An abstract class is declared using the abstract keyword. It may contain:

- Abstract methods (methods without a body)
- Concrete methods (methods with implementation)
- Constructors
- Instance variables
- Static and final methods

Syntax:

```
abstract class ClassName {  
  
    abstract void methodName(); // Abstract method  
  
    void concreteMethod() {    // Concrete method  
        System.out.println("Method implementation");  
    }  
}
```

Example: Abstract Class with Constructor and Methods.

```
import java.io.*;

abstract class Subject {
    Subject() {
        System.out.println("Learning Subject");
    }
    abstract void syllabus();
    void Learn(){
        System.out.println("Preparing Right Now!");
    }
}

class IT extends Subject {
    void syllabus(){
        System.out.println("C , Java , C++");
    }
}

class GFG {
    public static void main(String[] args) {
        Subject x=new IT();
        x.syllabus();
        x.Learn();
    }
}
```

Output: avinash

21

222.2

Example: Abstract Class with Partial Abstraction

```
abstract class Shape {
    String color;
    Shape(String color) { // Constructor
        this.color = color;
    }
    abstract double area(); // Abstract method
    void getColor() { // Concrete method
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    int radius;
    Circle(String color, int radius) {
        super(color);
        this.radius = radius;
    }
    double area() {
        return 3.14 * radius * radius;
    }
}

public class GFG{
    public static void main(String[] args) {
        Shape s = new Circle("Red", 5);
        s.getColor();
        System.out.println("Area: " + s.area());
    }
}
```

Example 3: The following version of the program declares area() as abstract inside Figure. This, of course, means that all classes derived from Figure must override area()

```
// Using abstract methods and classes.
abstract class Figure {
    double dim1;
    double dim2;
    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }
    // area is now an abstract method
    abstract double area();
}
class Rectangle extends Figure {
    Rectangle(double a, double b) {
        super(a, b);
    }
    // override area for rectangle
    double area() {
        System.out.println("Inside Area for Rectangle.");
        return dim1 * dim2;
    }
}
class Triangle extends Figure {
    Triangle(double a, double b) {
        super(a, b);
    }
    // override area for right triangle
    double area() {
```

```
System.out.println("Inside Area for Triangle.");
return dim1 * dim2 / 2;
}
}

class AbstractAreas {
public static void main(String args[]) {
// Figure f = new Figure(10, 10); // illegal now
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);
Figure figref; // this is OK, no object is created
figref = r;
System.out.println("Area is " + figref.area());
figref = t;
System.out.println("Area is " + figref.area());
}
}
```

As the comment inside **main()** indicates, it is no longer possible to declare objects of type **Figure**, since it is now abstract. And, all subclasses of **Figure** must override **area()**. To prove this to yourself, try creating a subclass that does not override **area()**. You will receive a compile-time error.

Although it is not possible to create an object of type **Figure**, you can create a reference variable of type **Figure**. The variable **figref** is declared as a reference to **Figure**, which means that it can be used to refer to an object of any class derived from **Figure**. As explained, it is through superclass reference variables that overridden methods are resolved at run time.

2.7 FINAL MEMBERS & CLASSES:

The final keyword is a non-access modifier used to restrict modification. It applies to variable for not changing its value, methods cannot be overridden and classes cannot be extended. It helps create constants, control inheritance and enforce fixed behavior.

The following are different contexts where the final is used:

1. Variable
2. Method
3. Class

Using final to Prevent Overriding

While method overriding is one of Java's most powerful features, there will be times when you will want to prevent it from occurring. To disallow a method from being overridden, specify **final** as a modifier at the start of its declaration. Methods declared as **final** cannot be overridden. The following fragment illustrates **final**:

```
class A {  
    final void meth() {  
        System.out.println("This is a final method.");  
    }  
}  
  
class B extends A {  
    void meth() { // ERROR! Can't override.  
        System.out.println("Illegal!");  
    }  
}
```

Because meth() is declared as final, it cannot be overridden in B. If you attempt to do so, a compile-time error will result.

The binding which can be resolved at compile time by the compiler is known as **static or early binding**. Binding of all the static, private and final methods is done at compile-time.

In the **late binding** or **dynamic binding**, the compiler doesn't decide the method to be called. Overriding is a perfect example of dynamic binding. In overriding both parent and child classes have the same method.

Example for Early Binding:

```
public class NewClass {  
    public static class superclass {  
        static void print() {  
            System.out.println("&quot;print in superclass.&quot;");  
        }  
    }  
    public static class subclass extends superclass {  
        static void print() {  
            System.out.println("&quot;print in subclass.&quot;");  
        }  
    }  
    public static void main(String[] args)  
    {  
        superclass A = new superclass();  
        superclass B = new subclass();  
        A.print();  
        B.print();  
    }  
}
```

Output:

print in superclass.

print in superclass.

Example for Late Binding:

```
public class NewClass {  
    public static class superclass {  
        void print() {  
            System.out.println("&quot;print in superclass.&quot;");  }  
        }  
    public static class subclass extends superclass {  
        @Override  
        void print() {  
            System.out.println("&quot;print in subclass.&quot;");  
        }  
    }  
    public static void main(String[] args)  
    { superclass A = new superclass();  
      superclass B = new subclass();  
      A.print();  
      B.print();  
    }  
}
```

Output:

print in superclass.

print in subclass.

Advantages of final Keyword

- Supports immutability by preventing reassignment
- Helps compiler and JVM optimize code in some scenarios
- Makes behavior predictable since values or methods stay unchanged
- Prevents accidental or unauthorized modification of critical logic
- Preserves API contracts by avoiding unwanted overriding

Using final to Prevent Inheritance

Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with **final**. Declaring a class as **final** implicitly declares all of its methods as **final**, too. As you might expect, it is illegal to declare a class as both **abstract** and **final** since an abstract class is incomplete by itself and relies upon its subclasses to provide complete implementations.

Here is an example of a **final** class:

```
final class A {  
  
    //...  
}  
  
// The following class is illegal.  
  
class B extends A { // ERROR! Can't subclass A  
  
    //...  
}
```

As the comments imply, it is illegal for **B** to inherit **A** since **A** is declared as **final**.

2.8 THE OBJECT CLASS

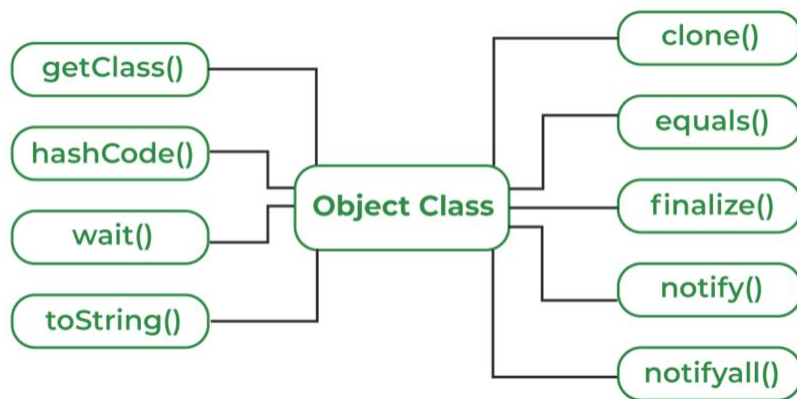
Object class (in java.lang) is the root of the Java class hierarchy. Every class in Java either directly or indirectly extends Object. It provides essential methods like toString(), equals(), hashCode(), clone() and several others that support object comparison, hashing, debugging, cloning and synchronization.

Why Object Class?

- Acts as the root of all Java classes
- Defines essential methods shared by all objects
- Provides default behavior for printing, comparing and cloning objects

- Supports thread communication (wait(), notify(), notifyAll())

Object Class Methods



1. toString() Method: toString() provides a String representation of an object and is used to convert an object to a String.

```
class Student{  
    String name = "Vishnu";  
    int age = 21;  
    @Override  
    public String toString(){  
        return "Student{name='" + name + "', age=" + age + "'";  
    }  
    public static void main(String[] args) {  
        Student s = new Student();  
        // Calls overridden toString()  
        System.out.println(s.toString());  
    }  
}
```

```
}
```

Output

```
Student{name='Vishnu', age=21}
```

Explanation: Overridden toString() prints a custom readable string for the object

2. hashCode() Method: hashCode() method returns the hash value of an object (not its memory address). Used heavily in hash-based collections like HashMap, HashSet, etc.

```
class Employee{  
    int id = 101;  
  
    @Override  
    public int hashCode(){  
        return id * 31; // Simple custom hash  
    }  
  
    public static void main(String[] args) {  
        Employee e = new Employee();  
        System.out.println(e.hashCode());  
    }  
}
```

Output

```
3131
```

Explanation: hashCode() returns an integer value used in hashing-based collections like HashMap. If two objects are equal, they must produce the same hash code..

3. equals(Object obj) Method

equals() method compares the given object with the current object. It is recommended to override this method to define custom equality conditions.

```
class Book{
```

```
String title;

Book(String title) {
    this.title = title;
}

@Override

public boolean equals(Object obj){
    Book b = (Book) obj;
    return this.title.equals(b.title);
}

public static void main(String[] args) {
    Book b1 = new Book("Java");
    Book b2 = new Book("Java");
    System.out.println(b1.equals(b2)); // true
}
}
```

Output

true

Explanation: equals() compares objects based on content rather than reference. Must be overridden when custom comparison logic is needed.

4. getClass() method: getClass() method returns the class object of "this" object and is used to get the actual runtime class of the object.

```
public class Hello{
    public static void main(String[] args)
    {
        Object o = new String("HelloWorld");
    }
}
```

```
Class c = o.getClass();

System.out.println("Class of Object o is: " + c.getName());

}

}
```

Output

Class of Object o is: java.lang.String

Explanation: The getClass() method is used to print the runtime class of the "o" object.

5. finalize() method: finalize() method is invoked by the Garbage Collector just before an object is destroyed. It runs when the object has no remaining references. You can override finalize() to release system resources and perform cleanup, but its use is discouraged in modern Java.

```
public class Geeks {

    public static void main(String[] args) {

        Geeks t = new Geeks();

        System.out.println(t.hashCode());

        t = null;

        // calling garbage collector

        System.gc();

        System.out.println("end");

    }

    @Override protected void finalize()

    {

        System.out.println("finalize method called");

    }

}
```

Output

1510467688

end

finalize method called

Explanation: The finalize() method is called just before the object is garbage collected.

6. clone() method: clone() method creates and returns a new object that is a copy of the current object.

```
class Student implements Cloneable{  
    int id = 1;  
    String name = "Vishnu";  
    @Override  
    public Object clone() throws CloneNotSupportedException{  
        return super.clone(); // shallow copy  
    }  
    public static void main(String[] args) throws Exception{  
        Student s1 = new Student();  
        Student s2 = (Student) s1.clone();  
        System.out.println(s1.name); // Vishnu  
        System.out.println(s2.name); // Vishnu  
    }  
}
```

Output

Vishnu

Vishnu

Explanation: clone() creates a copy of the current object (shallow copy by default). Class must implement Cloneable or else it throws CloneNotSupportedException.

7. Concurrency Methods: wait(), notify() and notifyAll(): These methods are related to thread Communication in Java. They are used to make threads wait or notify others in concurrent programming.

```
import java.io.*;

public class Book implements Cloneable {

    private String t; // title

    private String a; // author

    private int y;    // year

    public Book(String t, String a, int y)

    {

        this.t = t;

        this.a = a;

        this.y = y;

    }

    // Override the toString method

    @Override public String toString()

    {

        return t + " by " + a + " (" + y + ")";

    }

    // Override the toString method

    @Override public String toString()

    {

        return t + " by " + a + " (" + y + ")";

    }

    // Override the equals method
```

```
@Override public boolean equals(Object o)
{
    if (o == null || !(o instanceof Book)) {
        return false;
    }

    Book other = (Book)o;

    return this.t.equals(other.getTitle()) && this.a.equals(other.getAuthor()) && this.y ==
other.getYear();
}

// Override the hashCode method
@Override public int hashCode()
{
    int res = 17;

    res = 31 * res + t.hashCode();

    res = 31 * res + a.hashCode();

    res = 31 * res + y;

    return res;
}

// Override the clone method
@Override public Book clone()
{
    try {
        return (Book)super.clone();
    }

    catch (CloneNotSupportedException e) {
```

```
        throw new AssertionError();
    }
}

// Override the finalize method
@Override protected void finalize() throws Throwable
{
    System.out.println("Finalizing " + this);
}

public String getTitle() { return t; }
public String getAuthor() { return a; }
public int getYear() { return y; }
public static void main(String[] args)
{
    // Create a Book object and print its details
    Book b1 = new Book(
        "The Hitchhiker's Guide to the Galaxy",
        "Douglas Adams", 1979);
    System.out.println(b1);

    // Create a clone of the Book object and print its details
    Book b2 = b1.clone();
    System.out.println(b2);

    // Check if the two objects are equal
    System.out.println("b1 equals b2: " + b1.equals(b2));

    // Get the hash code of the two objects
    System.out.println("b1 hash code: " + b1.hashCode());
```

```
        System.out.println("b2 hash code: "+ b2.hashCode());  
        // Set book1 to null to trigger garbage collection and finalize method  
        b1 = null;  
        System.gc();  
    }  
}
```

Output

The Hitchhiker's Guide to the Galaxy by Douglas Adams (1979)

The Hitchhiker's Guide to the Galaxy by Douglas Adams (1979)

b1 equals b2: true

b1 hash code: 1840214527

b2 hash code: 1840214527

Explanation: The above example demonstrates the use of toString(), equals(), hashCode() and clone() methods in the Book class.

2.9 INTERFACES IN JAVA

An Interface in Java is an abstract type that defines a set of methods a class must implement.

- An interface acts as a contract that specifies what a class should do, but not how it should do it. It is used to achieve abstraction and multiple inheritance in Java. We define interfaces for capabilities (e.g., Comparable, Serializable, Drawable).
- A class that implements an interface must implement all the methods of the interface. Only variables are public static final by default.

Before Java 8, interfaces could only have abstract methods (no bodies). Since Java 8, they can also include default and static methods (with implementation) and since Java 9, private methods are allowed.

This example demonstrates how an interface in Java defines constants and abstract methods, which are implemented by a class.

Example Program:

```
import java.io.*;

// Interface Declared
interface testInterface {

    // public, static and final
    final int a = 10;

    // public and abstract
    void display();

}

// Class implementing interface
class TestClass implements testInterface {

    // Implementing the capabilities of Interface
    public void display(){

        System.out.println("Geek");

    }

}

class Geeks{

    public static void main(String[] args){

        TestClass t = new TestClass();

        t.display();

        System.out.println(t.a);

    }

}
```

Output

Geek

10

Note:

Private methods can only be called inside default or static methods.

Static methods are accessed using the interface name, not via objects.

To implement an interface, use the implements keyword.

Relationship Between Class and Interface

A class can extend another class and similarly, an interface can extend another interface. However, only a class can implement an interface and the reverse (an interface implementing a class) is not allowed

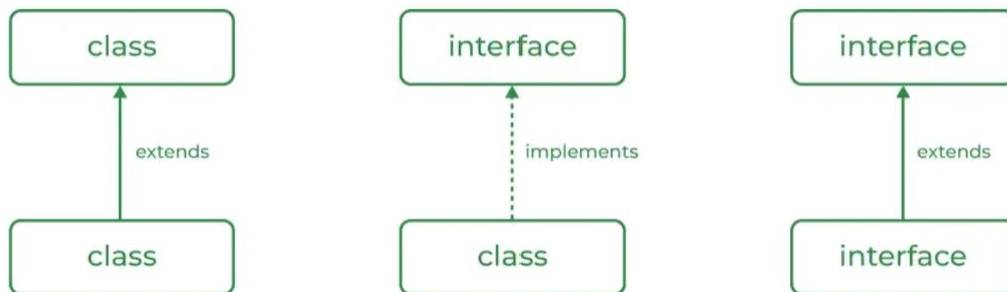


Figure: Relationship between Classes and Interfaces

When to Use Class and Interface?**Use a Class when:**

Use a class when you need to represent a real-world entity with attributes (fields) and behaviors (methods). Use a class when you need to create objects that hold state and perform actions. Classes are used for defining templates for objects with specific functionality and properties.

Use an Interface when: Use an interface when you need to define a contract for behavior that multiple classes can implement. Interface is ideal for achieving abstraction and multiple inheritance.

Let's consider the example of Vehicles like bicycles, cars and bikes share common functionalities, which can be defined in an interface, allowing each class (e.g., Bicycle, Car, Bike) to implement them in its own way. This approach ensures code reusability, scalability and consistency across different vehicle types.

```
import java.io.*;

interface Vehicle {

    // Abstract methods defined

    void changeGear(int a);

    void speedUp(int a);

    void applyBrakes(int a);

}

// Class implementing vehicle interface

class Bicycle implements Vehicle{

    int speed;

    int gear;

    // Change gear

    @Override

    public void changeGear(int newGear){

        gear = newGear;

    }

    // Increase speed

    @Override
```

```
public void speedUp(int increment){
    speed = speed + increment;
}

// Decrease speed

@Override

public void applyBrakes(int decrement){
    speed = speed - decrement;
}

public void printStates() {
    System.out.println("speed: " + speed + " gear: " + gear); }
}

class Main
{
    public static void main (String[] args)
    {
        // Instance of Bicycle(Object)

        Bicycle bicycle = new Bicycle();

        bicycle.changeGear(2);

        bicycle.speedUp(3);

        bicycle.applyBrakes(1);

        System.out.print("Bicycle present state : ");

        bicycle.printStates();

        // Instance of Bike (Object)

        Bike bike = new Bike();

        bike.changeGear(1);
```



```

        bike.speedUp(4);

        bike.applyBrakes(3);

        System.out.print("Bike present state : ");

        bike.printStates();

    }

}

```

Output

Bicycle present state : speed: 2 gear: 2

Bike present state : speed: 1 gear: 1

Multiple Inheritance in Java Using Interface: Java does not support multiple inheritance with classes to avoid ambiguity, but it supports multiple inheritance using interfaces.

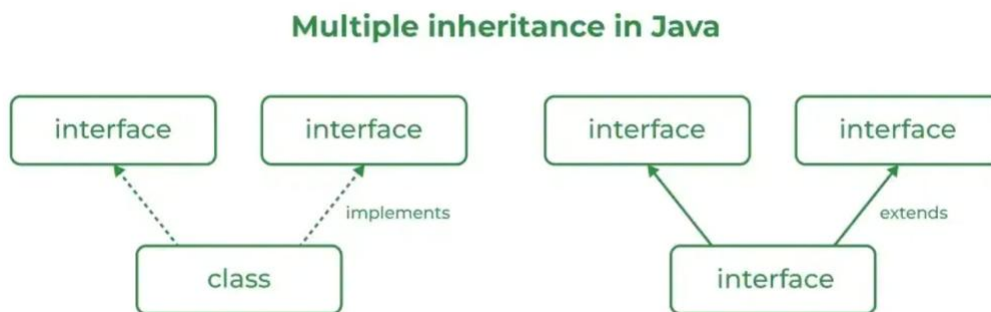


Figure: Multiple Inheritance in Java

```

import java.io.*;

// Add interface
interface Add{

    int add(int a,int b);

}

// Sub interface
interface Sub{

```

```
        int sub(int a,int b);
    }

// Calculator class implementing Add and Sub
class Cal implements Add , Sub
{
    // Method to add two numbers

    public int add(int a,int b){
        return a+b;
    }


    // Method to sub two numbers
    public int sub(int a,int b){
        return a-b;
    }
}

class GFG{
    // Main Method

    public static void main (String[] args){
        // instance of Cal class
        Cal x = new Cal();

        System.out.println("Addition : " + x.add(2,1));
        System.out.println("Substraction : " + x.sub(2,1));
    }
}
```

Output

Addition : 3

Substraction : 1

New Features Added in Interfaces in JDK 8

There are certain features added to Interfaces in JDK 8 update mentioned below:

1. Default Methods

Interfaces can define methods with default implementations.

Useful for adding new methods to interfaces without breaking existing implementations.

```
interface TestInterface
```

```
{
    final int a = 10;

    default void display() {
        System.out.println("hello");
    }
}
```

```
// A class that implements the interface.
```

```
class TestClass implements TestInterface
```

```
{
    // Driver Code

    public static void main (String[] args) {
        TestClass t = new TestClass();
        t.display();
    }
}
```

Output

hello

2. Static Methods

Interfaces can now include static methods. These methods are called directly using the interface name and are not inherited by implementing classes.

Another feature that was added in JDK 8 is that we can now define static methods in interfaces that can be called independently without an object. These methods are not inherited.

3. Functional Interface

Functional interfaces can be used with lambda expressions or method references. The `@FunctionalInterface` annotation can be used to indicate that an interface is a functional interface, although it's optional.

```
@FunctionalInterface
```

```
public interface Calculator {  
  
    int compute(int x, int y); // single abstract method  
  
}
```

New Features Added in Interfaces in JDK 9

From Java 9 onwards, interfaces can contain the following also:

1. Private Methods

Interfaces can now include private methods. Private methods are defined within the interface but it cannot be accessed by the implementing classes. Private methods cannot be overridden by implementing classes as they are not inherited.

```
interface Vehicle {  
  
    // Private method for internal use  
  
    private void startEngine() {  
  
        System.out.println("Engine started.");  
  
    }  
  
    // Default method that uses the private method  
  
    default void drive() {  
  
        // Calls the private method  
  
        startEngine();  
  
    }  
  
}
```

```
        System.out.println("Vehicle is now driving.");
    }
}

class Car implements Vehicle {

    // Car class implements Vehicle interface and inherits the default method 'drive'

}

public class Main {

    public static void main(String[] args) {

        Car car = new Car();

        // This will call the default method, which in turn calls the private method

        car.drive();

    }

}
```

Output

Engine started.

Vehicle is now driving.

Extending Interfaces

One interface can inherit another by the use of keyword extends. When a class implements an interface that inherits another interface, it must provide an implementation for all methods required by the interface inheritance chain.

```
interface A {

    void method1();

    void method2();

}

// B now includes method1 and method2

interface B extends A {
```

```
void method3();

}

// the class must implement all method of A and B.

class GFG implements B

{

    public void method1() {

        System.out.println("Method 1");

    }

    public void method2()

    {

        System.out.println("Method 2");

    }

    public void method3()

    {

        System.out.println("Method 3");

    }

    public static void main(String[] args)

    {

        // Instance of GFG class created

        GFG x = new GFG();

        // All Methods Called

        x.method1();

        x.method2();

        x.method3();

    }

}
```

Output*Method 1**Method 2**Method 3***Difference Between Class and Interface**

Although Class and Interface seem the same there are certain differences between Classes and Interface

Features	Class	Interface
Instantiation	In class, you can create an object.	In an interface, you can't create an object
Variables	Class can have instance variables	Variables are public static final (constants only).
Methods	Class can have concrete methods	In an interface, methods are abstract by default
Inheritance	It supports single inheritance	Supports multiple inheritance
Constructors	Can have constructors.	No constructors allowed.
Access Modifiers	Supports private, protected, public, default.	In an interface, all members are public by default
Keyword	Defined using class.	Defined using interface.
Default Methods	It does not support default methods	It supports default methods(JDK 8+)
Static Methods	Can have static methods.	Supports static methods (JDK 8+)
Private Methods	Can have private methods.	Supports private methods (JDK 9+).
Main Method	Can have main() for execution.	Can have main() (since JDK 8, as static methods are allowed).

2.10 PACKAGE FUNDAMENTALS

A package in Java is a mechanism to group related classes, interfaces, and sub-packages into a single unit. Packages help organize large applications, avoid naming conflicts, provide access protection, and make code modular and maintainable. A package in Java is a namespace that organizes a set of related classes and interfaces. It is known as a folder in a file system in which the related files (classes, interfaces, etc.) are stored together.

Java packages help in logically grouping classes and interfaces, making code more modular, readable, and easier to maintain.

- Avoiding name conflicts (two classes with the same name can exist in different packages)
- Providing access control using public, protected, and default access
- Reusability: packaged code can be imported and used anywhere
- Encouraging modular programming

Importance of Using Packages in Java:

- Code Organization and Maintainability: Packages group related classes, making your code more organized and easier to maintain. Large codebases are simpler to navigate and modify.
- Prevention of Naming Conflicts: By using packages, you can avoid conflicts between classes that have the same name but serve different purposes by isolating them in different namespaces.
- Enhanced Modularity and Reusability: Packages promote modular programming. You can design components in packages that can be reused across different projects without interference.
- Better Access Control and Security: Packages provide access modifiers (e.g., public, private) to control the visibility of classes, methods, and variables, enhancing security and encapsulation.

- **Simplified Maintenance and Updates:** With packages, you can easily update or modify specific parts of your program without affecting other unrelated parts, leading to more efficient and less error-prone maintenance.

Types of Java Packages:



Built-in Packages: Built-in Packages comprise a large number of classes that are part of the Java API. The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.

The library contains components for managing input, database programming, and much much more. The complete list can be found at Oracles website: <https://docs.oracle.com/javase/8/docs/api/>.

The library is divided into packages and classes. Meaning you can either import a single class (along with its methods and attributes), or a whole package that contain all the classes that belong to the specified package.

To use a class or a package from the library, you need to use the import keyword:

Syntax:

```
import package.name.Class; // Import a single class  
  
import package.name.*; // Import the whole package
```

Import a Class

If you find a class you want to use, for example, the Scanner class, which is used to get user input, write the following code:

Example:

```
import java.util.Scanner;
```

In the example above, java.util is a package, while Scanner is a class of the java.util package.

To use the Scanner class, create an object of the class and use any of the available methods found in the Scanner class documentation.

In our example, we will use the `nextLine()` method, which is used to read a complete line:

Example: Using the Scanner class to get user input:

```
import java.util.Scanner;

class Main {

    public static void main(String[] args) {

        Scanner myObj = new Scanner(System.in);

        System.out.println("Enter username");

        String userName = myObj.nextLine();

        System.out.println("Username is: " + userName);

    }

}
```

Import a Package

There are many packages to choose from. In the previous example, we used the Scanner class from the java.util package. This package also contains date and time facilities, random-number generator and other utility classes.

To import a whole package, end the sentence with an asterisk sign (*). The following example will import ALL the classes in the java.util package:

Example: `import java.util.*;`

Some of the commonly used built-in packages are:

- `java.lang`: Contains language support classes(e.g, classes that define primitive data types, math operations). This package is automatically imported.

- java.io: Contains classes for supporting input/output operations.
- java.util: Contains utility classes that implement data structures such as Linked Lists and Dictionaries, as well as support for date and time operations.
- java.applet: Contains classes for creating Applets.
- java.awt: Contains classes for implementing the components for graphical user interfaces.

The Java API is grouped into several core packages to support various programming needs, such as utility functions, input/output handling, networking, and GUI design.

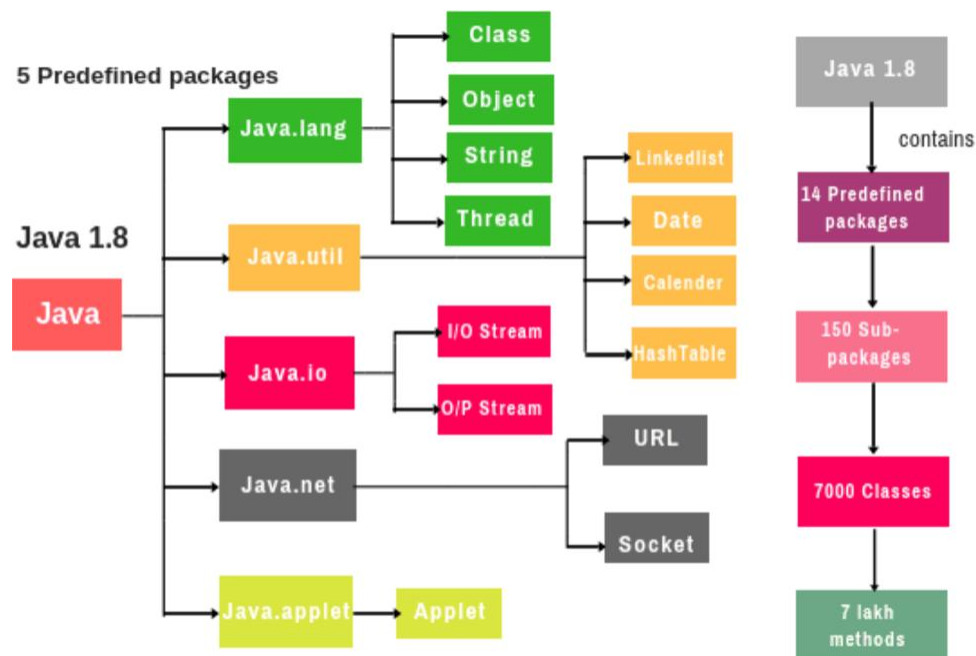


Fig: Predefined packages in Java

Example: Using java.util.Random (Built-in Package)

```
import java.util.Random; // built-in package

public class GFG{

    public static void main(String[] args) {

        // using Random class

        Random rand = new Random();

        // generates a number between 0–99

        int number = rand.nextInt(100);
```

```
        System.out.println("Random number: " + number);
    }
}
```

Example : java.util.Date and java.util.Calendar

```
import java.util.Date;
import java.util.Calendar;

public class UtilPackageDemo {

    public static void main(String[] args) {

        // Using Date class

        Date currentDate = new Date();

        System.out.println("Current Date and Time: " +currentDate);

        // Using Calendar class

        Calendar cal = Calendar.getInstance();

        System.out.println("Year: " + cal.get(Calendar.YEAR));

        System.out.println("Month: " + (cal.get(Calendar.MONTH) +1)); // 0-based

        System.out.println("Day: " +cal.get(Calendar.DAY_OF_MONTH));

    } }
```

Example : java.lang.Object & Class:

```
public class LangPackageDemo {

    public static void main(String [] args) {

        // Using Object class methods

        String name = "Java";

        System.out.println("Hash Code: " + name.hashCode());

        System.out.println ("ToString: " + name.toString());

        // Using Class class
```

```
Class<?> cls = name.getClass();

System.out.println("Class Name: " + cls.getName());

} }
```

Example : java.io InputStream and OutputStream:

```
import java.io.FileOutputStream;

import java.io.FileInputStream;

import java.io.IOException;

public class IOPackageDemo {

    public static void main(String[] args) {

        try {

            // Writing to file

            FileOutputStream fos = new FileOutputStream("sample.txt");

            String data = "Welcome to Java IO!";

            fos.write(data.getBytes());

            fos.close();

            // Reading from file

            FileInputStream fis = new FileInputStream("sample.txt");

            int i;

            System.out.print("File Content: ");

            while ((i = fis.read()) != -1) {

                System.out.print((char) i);

            }

            fis.close(); }

        catch (IOException e) {

            e.printStackTrace();

        }
```

```
}  
  
}  
  
}
```

User-defined Packages:

A user-defined package is a package created by the programmer to group related classes and interfaces. These packages are designed to group related classes and interfaces specific to an application or project. User-defined packages help developers avoid class name conflicts, organize their code, and increase the modularity and reusability of the codebase.

Creating a User-Defined Package

To create a user-defined package, you simply use the package keyword at the beginning of your Java file.

First We Should Choose A Name For The Package We Are Going To Create And Include. The package command In The first line in the java program source code. Further inclusion of classes, interfaces, annotation types, etc that is required in the package can be made in the package. For example, the below single statement creates a package name called “FirstPackage”.

Syntax: To declare the name of the package to be created. The package statement simply defines in which package the classes defined belong.

```
package FirstPackage ;
```

Implementation: To Create a Class Inside A Package

- First Declare The Package Name As The First Statement Of Our Program.
- Then We Can Include A Class As A Part Of The Package.

Example 1: Creating a Class Inside a Package

```
// Name of package to be created
```

```
package FirstPackage;
```

```
// Class in which the above created package belong to
```

```
class Welcome {  
    // main driver method  
    public static void main(String[] args)  
    {  
        // Print statement for the successful  
        // compilation and execution of the program  
        System.out.println(  
            "This Is The First Program Geeks For Geeks..");  
    }  
}
```

So Inorder to generate the above-desired output first do use the commands as specified use the following specified commands

Procedure to Generate Output:

1. Compile the Welcome.java file:

Command: javac Welcome.java

2. This command creates a Welcome.class file. To place the class file in the appropriate package directory, use:

Command: javac -d . Welcome.java

3. This command will create a new folder called FirstPackage. To run the class, use:

Command: java FirstPackage.Welcome

Output: The Above Will Give The Final Output Of The Example Program.

Example 2: Another Package Example

```
// Name of package to be created  
  
package data;  
  
// Class to which the above package belongs
```

```
public class Demo {  
    // Member functions of the class- 'Demo'  
    // Method 1 - To show()  
    public void show()  
    {  
        // Print message  
        System.out.println("Hi Everyone");  
    }  
    // Method 2 - To show()  
    public void view()  
    {  
        // Print message  
        System.out.println("Hello");  
    }  
}
```

Again, in order to generate the above-desired output first do use the commands as specified use the following specified commands

Procedure:

1. Compile the Demo.java file:

Command: javac Demo.java

2. This command will generate a Demo.class file:

Command: javac -d . Demo.java

3. This command will create a new folder called data containing the Demo.class file.

Note: In data Demo.java & Demo.class File should be present

Example 3: Accessing Data from Another Program


```
// Name of the package
import data.*;

// Class to which the package belongs
class ncj {

    // main driver method
    public static void main(String arg[])
    {
        // Creating an object of Demo class
        Demo d = new Demo();

        // Calling the functions show() and view()
        // using the object of Demo class
        d.show();
        d.view();
    }
}
```

Procedure to Generate the Output:

1. Compile the ncj.java file:

Command: javac ncj.java

2. The above command compiles ncj.java and requires the Demo.class file to be present in the data package.

Command: java ncj

// To Run This File

Output: Generated on the terminal after the above command is executed

Hi Everyone

Hello

Access protection in java packages

In java, the access modifiers define the accessibility of the class and its members. For example, private members are accessible within the same class members only. Java has four access modifiers, and they are default, private, protected, and public.

In java, the package is a container of classes, sub-classes, interfaces, and sub-packages. The class acts as a container of data and methods. So, the access modifier decides the accessibility of class members across the different packages.

In java, the accessibility of the members of a class or interface depends on its access specifiers. The following table provides information about the visibility of both data members and methods.

Access of class and Interfaces in Java:

Access Specifier \ Accessibility Location	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

🔔 The **public** members can be accessed everywhere.

🔔 The **private** members can be accessed only inside the same class.

🔔 The **protected** members are accessible to every child class (same package or other packages).

🔔 The **default** members are accessible within the same package but not outside the package.

Default Access (Package-Private)

When **no access modifier** is used, it is **accessible only within the same package**.

```
package pack1;
```

```
class A {  
    int x = 10; // default access  
}
```

✓ Accessible inside **pack1**

✗ Not accessible outside **pack1**

Public Access

Accessible **from anywhere**, inside or outside the package.

```
package pack1;  
  
public class A {  
    public int x = 10;  
}
```

✓ Same package

✓ Different package

✓ Subclasses

✓ Non-subclasses

Protected Access (Important for Packages)

Protected members are : Accessible within the same package, Accessible in subclasses outside the package

```
package pack1;  
  
public class A {  
    protected int x = 10;  
}
```

```
package pack2;

import pack1.A;

class B extends A {

    void show() {

        System.out.println(x); // allowed

    }

}
```

✗ Cannot be accessed by **non-subclass** in another package.

Private Access

Accessible **only within the same class**.

```
class A {

    private int x = 10;

}
```

Package-Level Protection

→ Default and protected access provide **package-level protection**

- Prevents unauthorized access from outside packages
- Helps in **encapsulation**
- Useful when building **large applications**

2.11 REFLECTIONS IN JAVA

Reflection is a feature in the Java programming language. It allows an executing Java program to examine or "introspect" upon itself, and manipulate internal properties of the program. For example, it's possible for a Java class to obtain the names of all its members and display them.

The ability to examine and manipulate a Java class from within itself may not sound like very much, but in other programming languages this feature simply doesn't exist. For example, there is

no way in a Pascal, C, or C++ program to obtain information about the functions defined within that program.

One tangible use of reflection is in JavaBeans, where software components can be manipulated visually via a builder tool. The tool uses reflection to obtain the properties of Java components (classes) as they are dynamically loaded.

A Simple Example: To see how reflection works, consider this simple example:

```
import java.lang.reflect.*;

public class DumpMethods {

    public static void main(String args[])

    {

        try {

            Class c = Class.forName(args[0]);

            Method m[] = c.getDeclaredMethods();

            for (int i = 0; i < m.length; i++)

                System.out.println(m[i].toString());

        }

        catch (Throwable e) {

            System.err.println(e);

        }

    }

}
```

For an invocation of:

```
java DumpMethods java.util.Stack
```

the output is:

```
public java.lang.Object java.util.Stack.push(
    java.lang.Object)
public synchronized
```

```
java.lang.Object java.util.Stack.pop()

public synchronized

java.lang.Object java.util.Stack.peek()

public boolean java.util.Stack.empty()

public synchronized

int java.util.Stack.search(java.lang.Object)
```

That is, the method names of class `java.util.Stack` are listed, along with their fully qualified parameter and return types.

This program loads the specified class using `Class.forName`, and then calls `getDeclaredMethods` to retrieve the list of methods defined in the class. `java.lang.reflect.Method` is a class representing a single class method.

Setting Up to Use Reflection

The reflection classes, such as `Method`, are found in `java.lang.reflect`. There are three steps that must be followed to use these classes. The first step is to obtain a `java.lang.Class` object for the class that you want to manipulate. `java.lang.Class` is used to represent classes and interfaces in a running Java program.

One way of obtaining a Class object is to say:

`Class c = Class.forName("java.lang.String");` to get the `Class` object for `String`. Another approach is to use: `Class c = int.class;` or `Class c = Integer.TYPE;` to obtain `Class` information on fundamental types. The latter approach accesses the predefined `TYPE` field of the wrapper (such as `Integer`) for the fundamental type.

The second step is to call a method such as `getDeclaredMethods`, to get a list of all the methods declared by the class. Once this information is in hand, then the third step is to use the reflection API to manipulate the information. For example, the sequence:

```
Class c = Class.forName("java.lang.String");

Method m[] = c.getDeclaredMethods(); System.out.println(m[0].toString());
```

will display a textual representation of the first method declared in String.

In the examples below, the three steps are combined to present self contained illustrations of how to tackle specific applications using reflection.

Simulating the instanceof Operator

Once Class information is in hand, often the next step is to ask basic questions about the Class object. For example, the Class.isInstance method can be used to simulate the instanceof operator:

```
class A {}

public class instance1 {
    public static void main(String args[])
    {
        try {
            Class cls = Class.forName("A");
            boolean b1
                = cls.isInstance(new Integer(37));
            System.out.println(b1);
            boolean b2 = cls.isInstance(new A());
            System.out.println(b2);
        }
        catch (Throwable e) {
            System.err.println(e);
        }
    }
}
```

In this example, a Class object for A is created, and then class instance objects are checked to see whether they are instances of A. Integer(37) is not, but new A() is.

Finding Out About Methods of a Class

One of the most valuable and basic uses of reflection is to find out what methods are defined within a class. To do this the following code can be used:

```
import java.lang.reflect.*;

public class method1 {
    private int f1(
        Object p, int x) throws NullPointerException
    {
        if (p == null)
            throw new NullPointerException();
        return x;
    }
    public static void main(String args[])
    {
        try {
            Class cls = Class.forName("method1");
            Method methlist[]
                = cls.getDeclaredMethods();
            for (int i = 0; i < methlist.length; i++) {
                Method m = methlist[i];
                System.out.println("name = " + m.getName());
                System.out.println("decl class = " + m.getDeclaringClass());
                Class pvec[] = m.getParameterTypes();
                for (int j = 0; j < pvec.length; j++)
                    System.out.println("param #" + j + " " + pvec[j]);
                Class evec[] = m.getExceptionTypes();
                for (int j = 0; j < evec.length; j++)
                    System.out.println("exc #" + j + " " + evec[j]);
                System.out.println("return type = " + m.getReturnType());
            }
        }
    }
}
```

```
        System.out.println("-----");
    }
}
catch (Throwable e) {
    System.err.println(e);
}
}
```

The program first gets the Class description for method1, and then calls `getDeclaredMethods` to retrieve a list of Method objects, one for each method defined in the class. These include public, protected, package, and private methods. If you use `getMethods` in the program instead of `getDeclaredMethods`, you can also obtain information for inherited methods.

Once a list of the Method objects has been obtained, it's simply a matter of displaying the information on parameter types, exception types, and the return type for each method. Each of these types, whether they are fundamental or class types, is in turn represented by a Class descriptor. The output of the program is:

```
name = f1
decl class = class method1
param #0 class java.lang.Object
param #1 int
exc #0 class java.lang.NullPointerException
return type = int
-----
name = main
decl class = class method1
param #0 class [Ljava.lang.String;
return type = void
```

Module Question Paper

1. What is inheritance in Java? Explain with an example. What are the advantages of inheritance in object-oriented programming?
2. Explain the different types of inheritance supported in Java? Why does Java not support multiple inheritance using classes?
3. What is the purpose of the super keyword? Write a Java program where class Animal is inherited by class Dog and demonstrate method inheritance.
4. Create a class Vehicle with properties speed and color. Derive a class Car and display both properties.
5. Demonstrate multilevel inheritance using three classes: Person → Employee → Manager
6. What is method overriding in Java? What are the rules for method overriding?
7. What is the difference between method overriding and method overloading?
8. What happens if the overridden method is declared final? Can static methods be overridden? Explain.
9. Write a Java program demonstrating method overriding using Shape → Circle classes.
10. Create a class Bank with method getInterestRate() and override it in subclasses SBI, ICICI, and HDFC.
11. What are annotations in Java? What is the purpose of @Override annotation? Explain @Deprecated annotation with an example.
12. What is @SuppressWarnings used for? What are the types of annotations in Java? Write a Java program demonstrating the use of @Override.
13. Explain how custom annotations are created in Java.
14. What is a static variable and static method? Can static methods access instance variables? Why or why not?
15. What is a static block in Java? What is the difference between static and non-static members?
16. Write a Java program demonstrating the use of a static variable to count objects created.
17. Create a class with a static method that prints a message without creating an object.
18. What is an inner class in Java? Explain the types of inner classes.
19. What is the difference between a static nested class and a non-static inner class?

20. What is a local inner class? What is an anonymous inner class? Write a Java program demonstrating a member inner class.
 21. Write a Java program using an anonymous inner class to implement an interface.
 22. What is an abstract class in Java? What is an abstract method. Can an abstract class have constructors? Explain. Can abstract classes contain non-abstract methods?
 23. What is the difference between abstract class and interface? Write a Java program using abstract class Shape with abstract method area().
 24. Create an abstract class Employee and implement it in subclasses Manager and Clerk.
 25. What is a final variable and final class in Java? What is a final method? Why is the String class declared final?
 26. What happens if a class is declared final? Write a program demonstrating a final variable. Create a final class and attempt to inherit it. What happens?
 27. What is the Object class in Java? Why is the Object class considered the root class of Java? Explain the toString() method. What is the purpose of the equals() method? What is the hashCode() method?
 28. Write a program overriding toString() method and demonstrating equals() method.
 29. What is an interface in Java? What are the features of interfaces?
 30. What is the difference between an interface and an abstract class? Can a class implement multiple interfaces? Explain. What are default methods in interfaces?
 31. Write a Java program implementing an interface Printable.
 32. Create an interface Shape and implement it in classes Circle and Rectangle.
 33. What is a package in Java? What are the advantages of packages? What is the difference between built-in packages and user-defined packages?
 34. How do you create a package in Java? What is the purpose of the import statement?
 35. Write a Java program creating a user-defined package. And also write a program importing a specific class from a package.
 36. What is reflection in Java? What is the purpose of the Class class? How can reflection be used to inspect methods of a class? What are the advantages of reflection?
 37. What are the limitations of reflection? Write a Java program that prints all methods of a class using reflection.
 38. Write a program to dynamically create an object using reflection.
-